



5

31

4650sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Games

د. باسل الخطيب



RB Informatics; 05/11/2025

خوارزميات البحث الذكية

مقدمة:

كنا قد تحدثنا في المحاضرات السابقة عن خوارزميات البحث الذكية العادية كان لدينا Agent واحد (single player) يحاول الوصول للهدف هو فقط الذي يقرر المسار ولا يوجد خصم يعاند حركة هذا اللاعب في كل خطوة.

اليوم سنتحدث عن الـ Game Search التي تندرج تحت نوع الـ adversarial search هنا لدينا خصم أي يوجد عميلان (multiplayer)، وقراراته تؤثر على مسار اللاعب الآخر، هنا نحن لا نبحث عن مسار للهدف بل نبحث عن قرار move يضمن أفضل نتيجة ممكنة حتى لو كان الخصم يلعب بأفضل طريقة ضدنا، أي لا يوجد خطة واحدة تضمن الفوز بل خطة احتمالات تكون ردود لقرارات الخصم (إذا لعب الخصم كذا أنا بلعب كذا وكذا).

مدخل للألعاب وأنواعها :

Deterministic		chance
Perfect information	Chess, Checkers, Go, Othello	Backgammon, Monopoly
Imperfect information	Battleship	Bridg, Poker, Scabble, Nuclear war

1. Deterministic :

حتمية، أي لا يوجد عشوائية تعتمد على الحظ

2. Chance:

عشوائية تعتمد على الحظ

3. Perfect information:

كل شيء ظاهر للطرفين مثل (شطرنج)

4. Imperfect information :

في معلومات مخفية باللعبة مثل (البوكر).

نوع الألعاب التي سنهتم بدراستها هي ال Deterministic + Perfect info وهنا تكون الخوارزمية المستخدمة هي ال Minimax

في هذه الألعاب:

اللاعبين يتناوبون (move in turn)، كل لاعب يرى ما يستطيع خصمه أن يلعبه وما لعب. **النتيجة** هي فوز **إو** خسارة **أو** تعادل (مكسب لاعب هو خسارة اللاعب الآخر)

EX: Grundy's Game

بفرض لدينا رتل (برج) من 7 قطعة نقدية ويوجد لاعبين يلعبان بشكل متناوب يمكن للاعب في دوره أن يقسم الرتل إلى رتلين بعدد قطع غير متساوي تنتهي اللعبة عندما لا يستطيع اللاعب تقسيم الرتل الذي امامه مثلاً: لو كان لدي الرتل الذي امام اللاعب الحالي مكون من 2 قطعة هنا يخسر اللاعب لأنه لا يستطيع تقسيم هذا الرتل إلى رتلين بعدد قطع غير متساوي. باعتبار أسماء اللاعبين Min ،Max (سنشرح سبب التسمية بالفقرة القادمة)

الآن السؤال: هل يمكننا وضع استراتيجية واضحة وصريحة لفوز أحد اللاعبين بفرض Max ؟

■ أولاً لنفكر بشكل عادي ما هي اللعبة وكيف يمكن لعبها؟

بفرض أننا اللاعب ماكس ونريد الفوز بالبداية كيف نلعب؟

هدفنا تقسيم الرتل المواجه بفرض كنا في حالة البداية فالرتل مكون من 7 قطع لنا إلى قسمين غير متساويين مثلاً: { 6 , 1 } او { 5 , 2 } او { 3,4 }.

نبدأ من أصغر رقم بالـ 7 وهو 1 ونضعه برتل والباقي 6 نضعه بالرتل الآخر ونكمل إلى أن نصل $(n/2 - 1)$

■ الآن الخصم سوف يأخذ المجموعات التي أمامه ويحاول أن يقوم بتقسيمها مثلاً المجموعة { 3 , 4 } أولاً الرتل 4 يقسمه إلى { 3 , 1 } أي أصبحت المجموعة الناتجة { 3 , 3 , 1 } الآن يكمل البرج 3 ويقسمه إلى { 2 , 1 } أصبحت المجموعة هي { 4 , 2 , 1 } .

■ الآن نحن نأخذ المجموعة { 3 , 3 , 1 } نأخذ البرج 3 ونقسمه إلى { 1,2 } أصبحت المجموعة { 3,2,1,1 }

انتبه! في كل دور يسمح بأخذ برج واحد أي هنا الـ 3 نغيرها مرة واحدة فقط

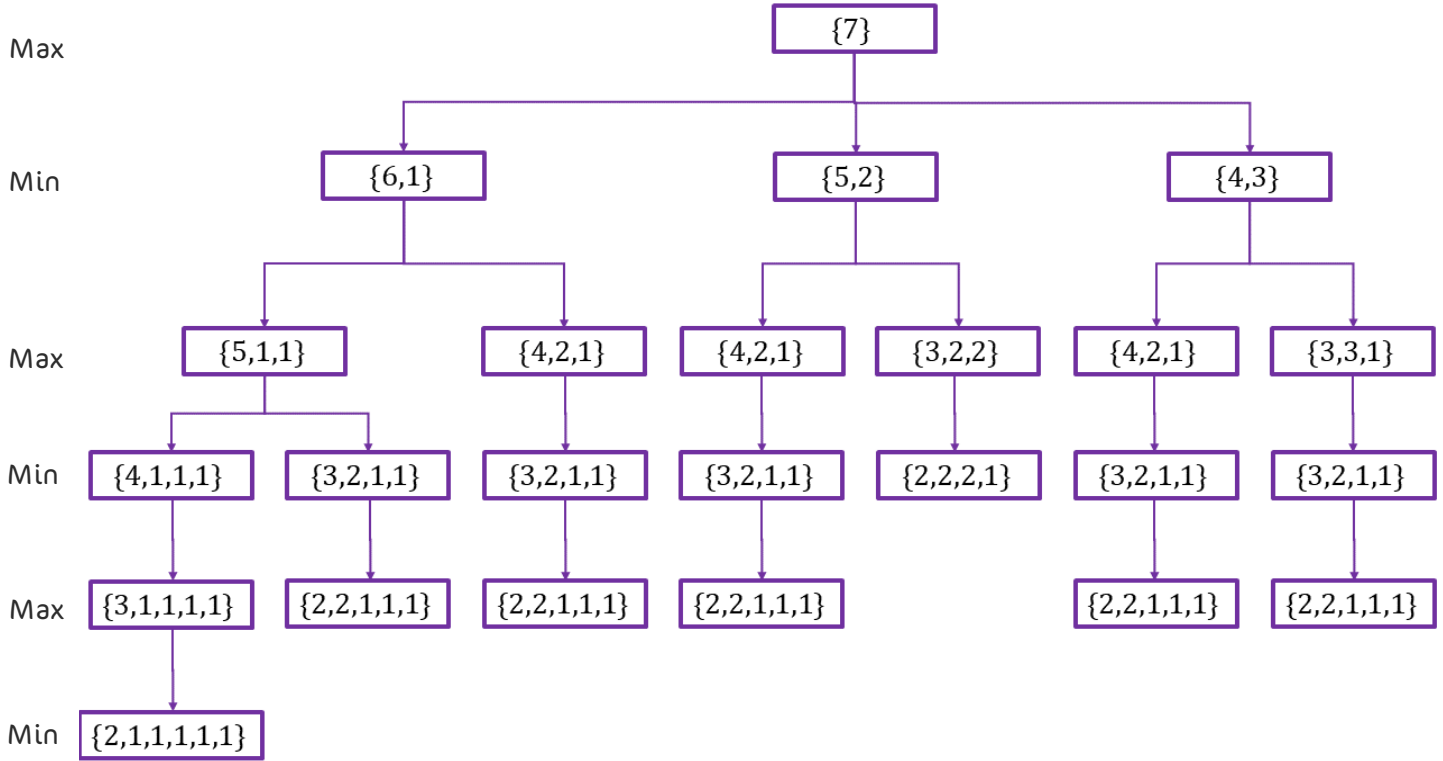
■ الآن نكمل ونأخذ المجموعة الثانية { 4 , 2 , 1 } نقسم البرج 4 إلى { 3 , 1 } لتصبح المجموعة { 3,2,1,1 }

■ الآن دور الخصم يأخذ المجموعة { 3,2,1,1 } ويقوم بتقسيم الـ 3 إلى { 2 , 1 } لتصبح النتيجة { 2,2,1,1,1 }

■ الآن دورنا لم يتبقى أي برج يمكننا القيام بقسمته بالتالي خسرنا مع الأسف

الآن بعد أن فهمنا مبدأ اللعبة لنرسم الشجرة المعبرة عن كل الاحتمالات الممكنة:

- بما أننا اللاعب max نلاحظ أننا نخسر في أغلب الحالات لأننا نصل إلى مراحل لا نستطيع فيها التقسيم أي تكون فيها المجموعات من الشكل {2,2,1,1,1} او من الشكل {2,1,1,1,1}



حسناً ماذا نفعل بما أننا اللاعب max يجب ان نفكر بطريقة لمعرفة كيف يمكن أن نفوز ؟

ملاحظة:

- اللعبة = شجرة البحث
- الحالة (state) = هي تقسيمات الارتال
- الحركات (move) = تقسيم الرتل إلى رتلين غير متساويين
- النهاية = عندما تصبح الارتال 1 او 2
- الخسارة = اللاعب الذي لا يستطيع الحركة

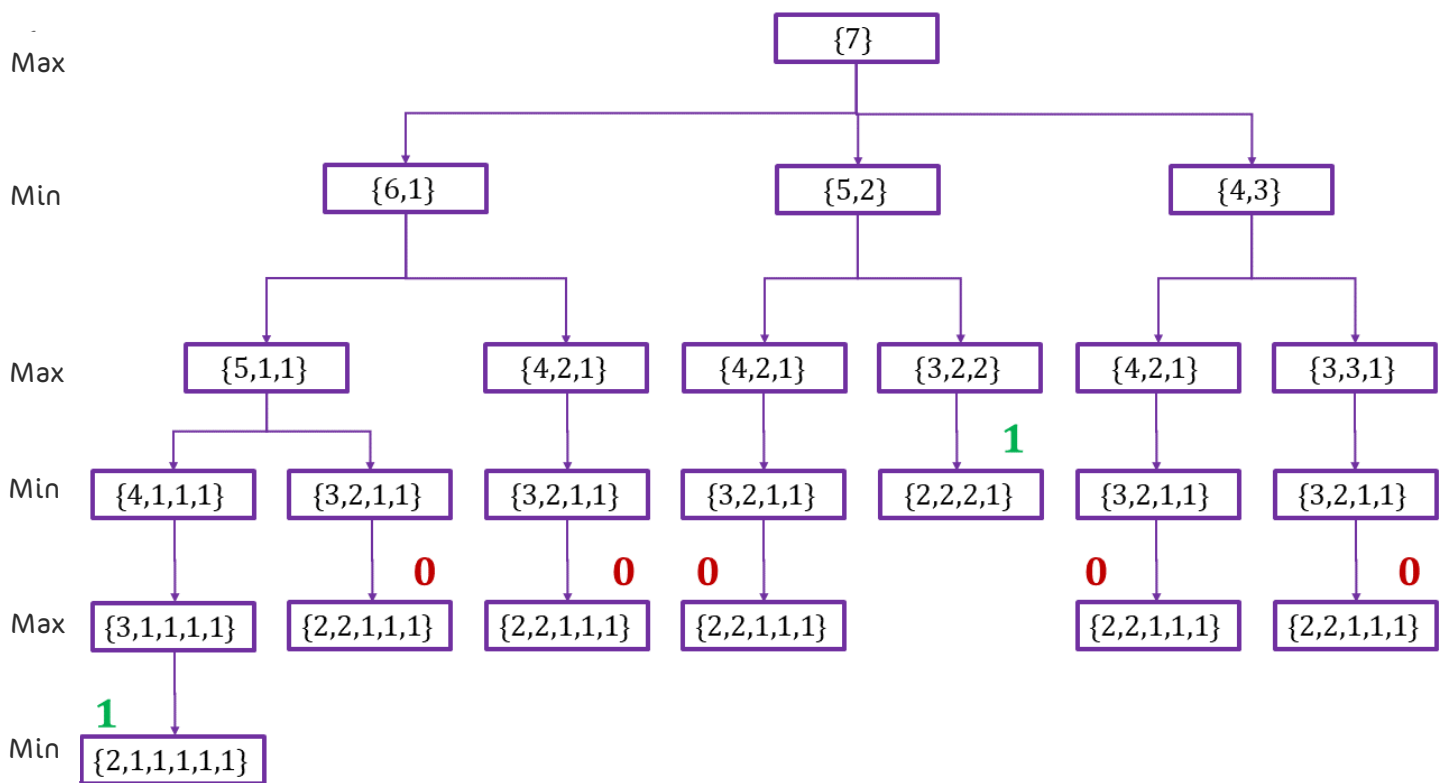
ملاحظة:

في الـ search tree كل عقدة هي ممكن أن تكون الهدف أما في الـ game tree كل عقدة هي نهاية تعطي نتيجة win/loss/draw

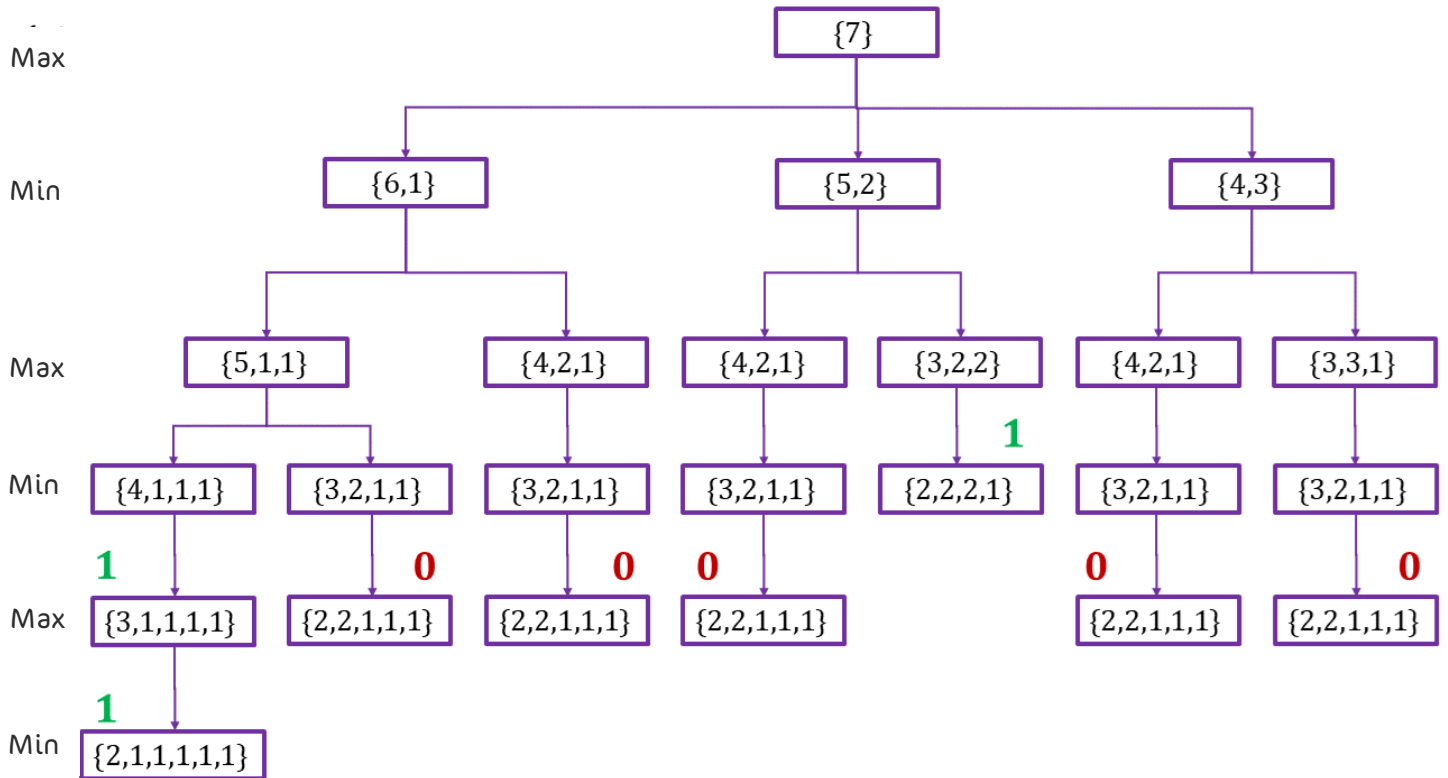
■ عند العقدة التي يفوز اللاعب max عندها نضع القيمة 1
وعند الحالة التي يفوز اللاعب min عندها نضع القيمة 0 (يخسر اللاعب max)

■ نفرض أن كلا اللاعبين يلعبان بشكل أمثلي
والحالات التي نعرف ماذا نضع قيمة لها هي الأوراق حتماً لأن اللعبة تنتهي عندها بخسارة احد اللاعبين
ننتبه! ان في هذه اللعبة لا يوجد تعادل .

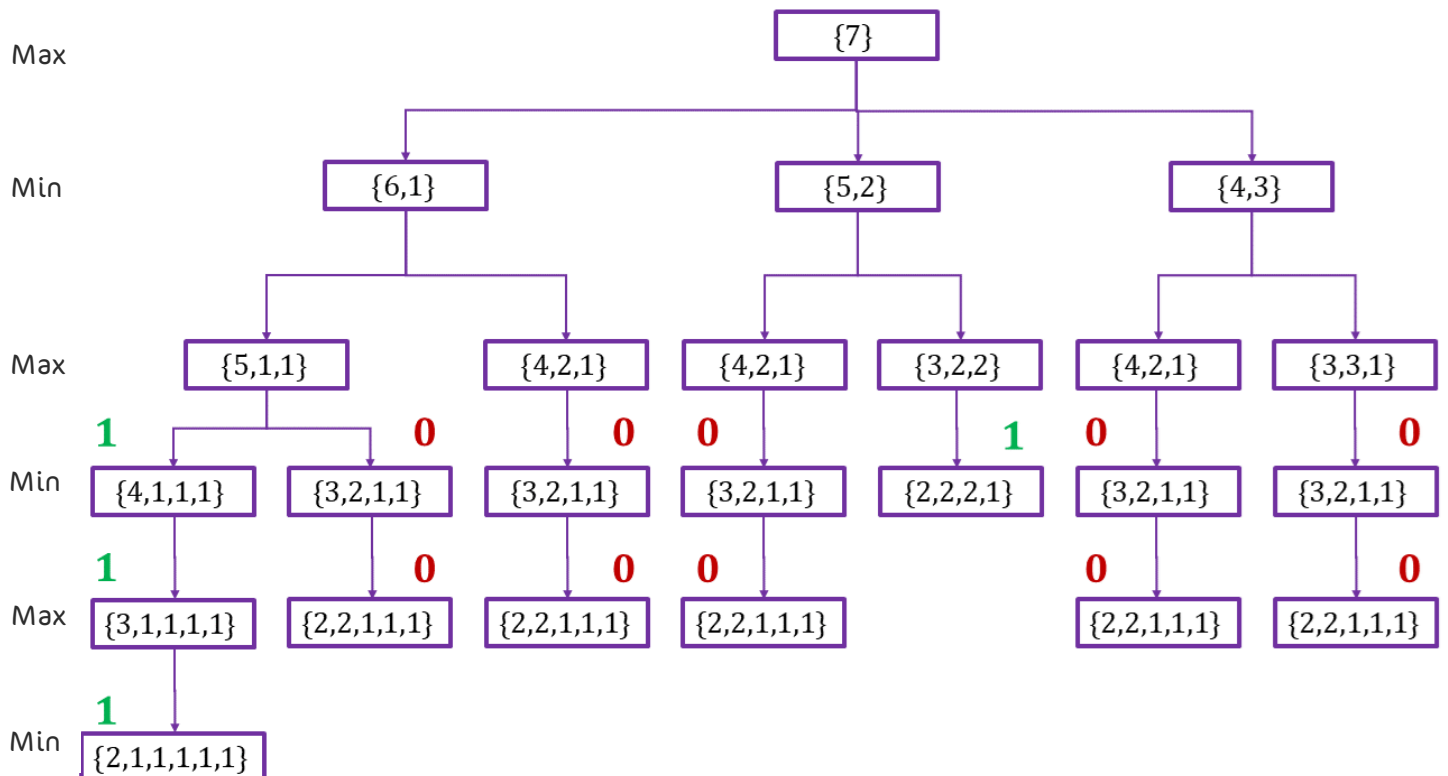
1. نبدأ من الأوراق بعملية إعطاء القيم ليصبح شكل الشجرة كالتالي:



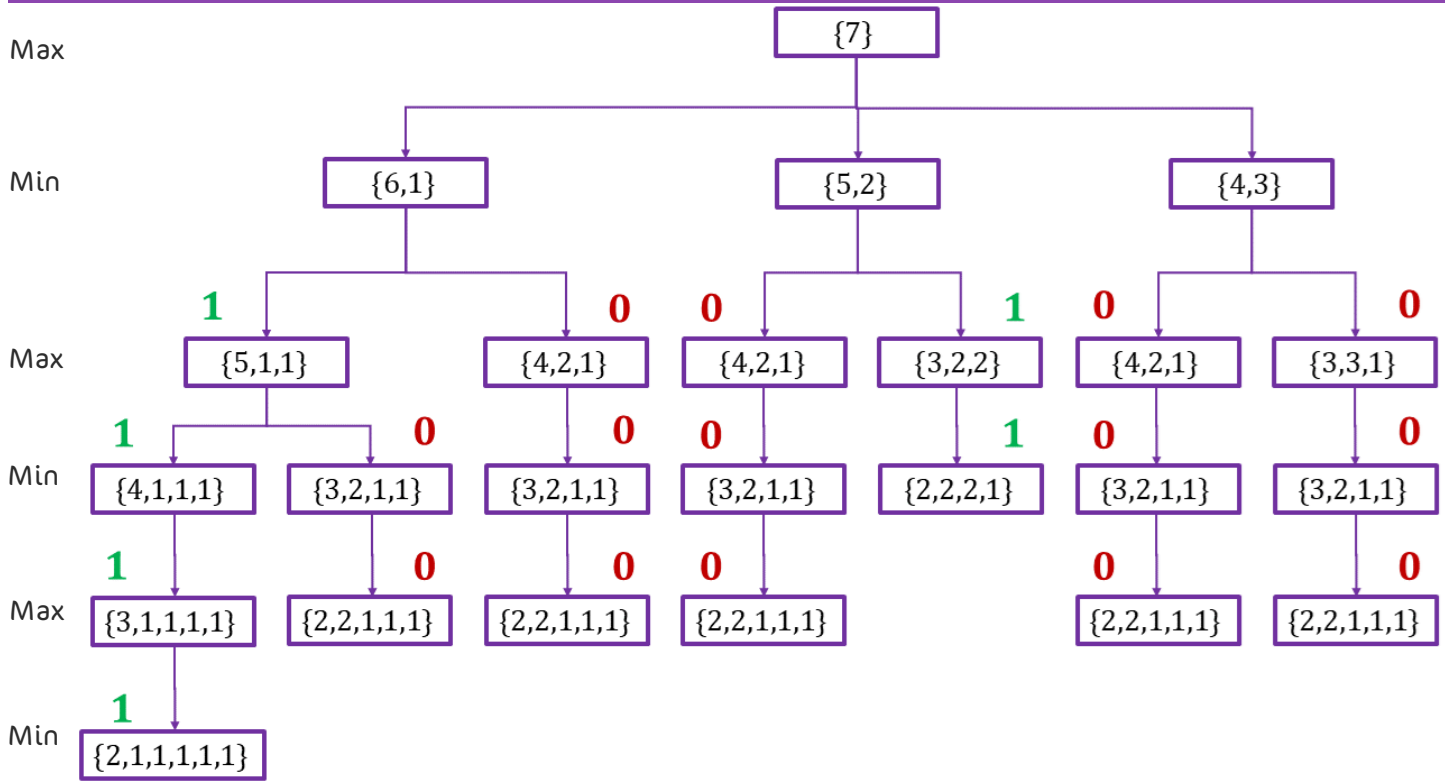
2. بعد أن انتهينا من أدنى مستوى ننتقل إلى المستوى الأعلى ، بما أن الحالة $\{2,1,1,1,1\}$ رابعة وهي ابن وحيد للحالة $\{3,1,1,1,1\}$ إذن هي أيضاً رابعة



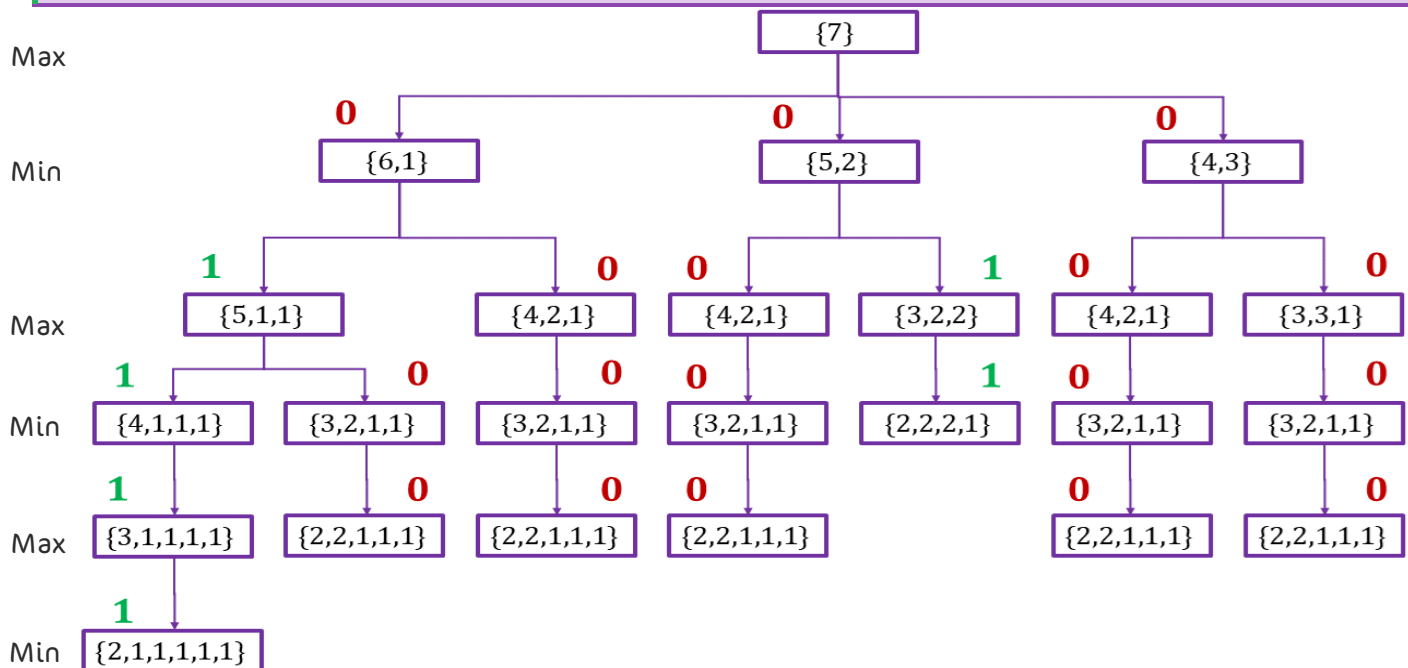
نلاحظ أن المستوى قبل الأخير أصبحنا نملك جميع قيمه، بنفس الطريقة ننتقل لبناء المستوى السابق،
نلاحظ أن لكل عقدة من المستوى السابق أيضاً ابن واحد فليس هنالك أي عملية تفضيل، فتصبح الشجرة:



الآن وصلنا للمستوى الثالث نلاحظ أن العقدة $\{5,1,1\}$ لديها ابنين أحدهما قيمته 0 (اللاعب max سيخسر لو سلك هذا الطريق) والآخر قيمته 1 (اللاعب max سيربح لو سلك هذا الطريق)، فنحن بحاجة لمعرفة ما القرار الأمثل الذي سيتخذه اللاعب الذي دوره الآن، نلاحظ أنه دور اللاعب max فهو بالتأكيد سيختار الخيار صاحب القيمة 1 حتى يفوز هو باللعبة، إذًا قيمة العقدة تصبح 1 أما باقي العقد فليس لها سوى ابن واحد فستأخذ نفس قيمته.

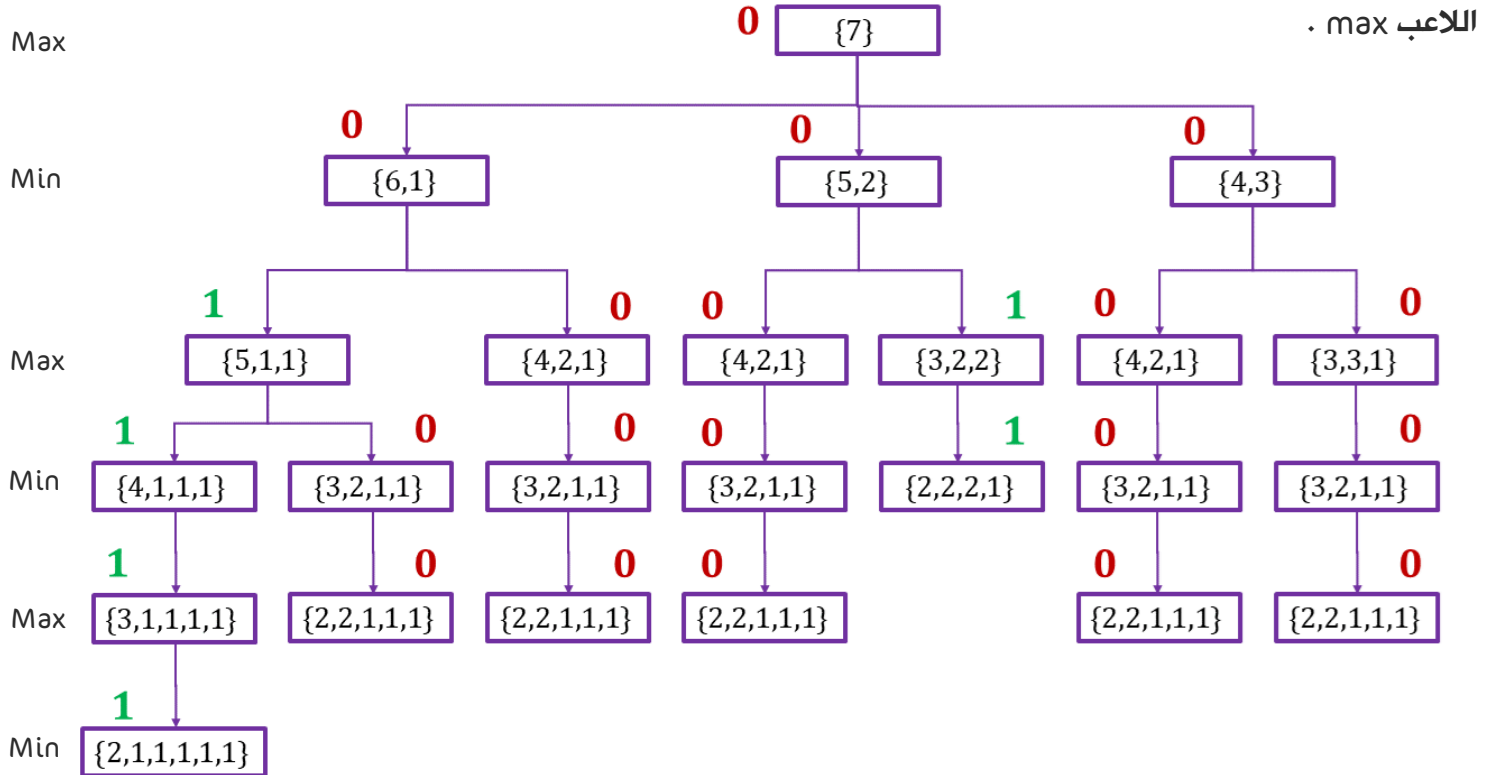


الآن وصلنا إلى المستوى الثاني نلاحظ أن جميع العقد لها أكثر من ابن، ففي جميع العقد نحن بحاجة لاتخاذ القرار، وبما أنه دور اللاعب min فهو سيحاول جعل اللاعب max يخسر، أي إذا كان هنالك ضمن الخيارات 0 فسيختاره، وإلا فهو سيكون مجبراً على اختيار 1 وترك اللاعب الأول يفوز، فتصبح القيم:



والآن لنستنتج قيمة عقدة المستوى الأول ونكتشف نتيجة اللعبة:

نلاحظ أن للعقدة أكثر من ابن فهي نقطة اتخاذ قرار، بما أنه دور اللاعب الأول فسيسعى لاختيار الطريق الذي يجعله يربح إن أمكن، أي إن كان هنالك 1 فسيختاره، وإلا فهو مجبر على اختيار 0، وهنا سيجبر على اختيار 0 وسيخسر



الآن يا أصدقائي نلاحظ أنه في حال نحن من بدأنا (اللاعب max) باللعبة فنسبة خسارتنا سوف تكون كبيرة جداً لذلك ندع الخصم اللاعب min هو من يبدأ باللعبة

إذا ماهي الإستراتيجية الراجعة للعبة max ؟

- إذا الدور للاعب MIN حتى يضمن MAX الفوز، يجب أن تكون كل حركات MIN تؤدي لنهايات MAX يربح فيها (قيمتها واحد) لأن MIN سيختار الأسوأ لـ MAX لذلك في حال كانت الخيارات امامه هي واحد سيكون مجبراً على اختيار هذه القيمة وبالتالي سيفوز لـ MAX
- إذا الدور للاعب MAX يكفي وجود حركة واحدة تؤدي لنهاية جيدة (القيمة واحد كما في مثالنا) لـ MAX لأنه سيختار الأفضل لنفسه

وهذه بالضبط سبب تسمية الخوارزمية بـ minimax :

- عند عقد MAX نأخذ أفضل ابن.
- عند عقد MIN نأخذ أسوأ ابن لـ MAX

كيف نقوم بتمثيل اللعبة في خوارزمية الـ minimax:

state:

تعبير عن وضع اللعبة حالياً بالإضافة الى دور أي لاعب.

Operators:

الحركات التي يسمح للاعب القيام بها في دوره

Terminal test:

اختبار هل اللعبة انتهت؟

Evaluation Function:

رقم يمثل قيمة الحالة (في الأوراق النهائية).

مثلاً: +1 فوز و -1 خسارة و 0 تعادل.

ملاحظة: الخوارزمية تفترض دوماً أن الخصم يختار الحركة التي تزيد مصلحته أي تقلل مصلحة اللاعب الآخر هذا الافتراض هو ما يميز الخوارزمية فنحن لا نخطط ضد خصم ضعيف بل ضد أفضل خصم ممكن

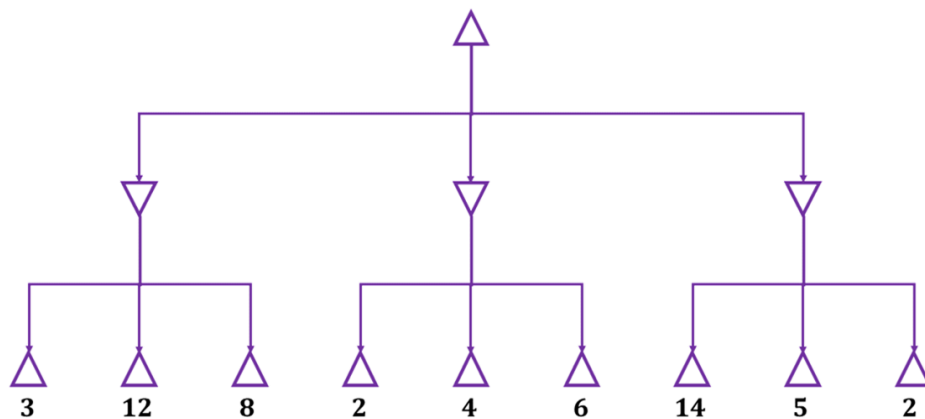
مثال 2:

في هذا المثال سنوضح كيفية عمل الخوارزمية minimax للألعاب الـ Deterministic + Perfect Information بفرض لدينا المخطط التالي الذي يمثل لعبة ما

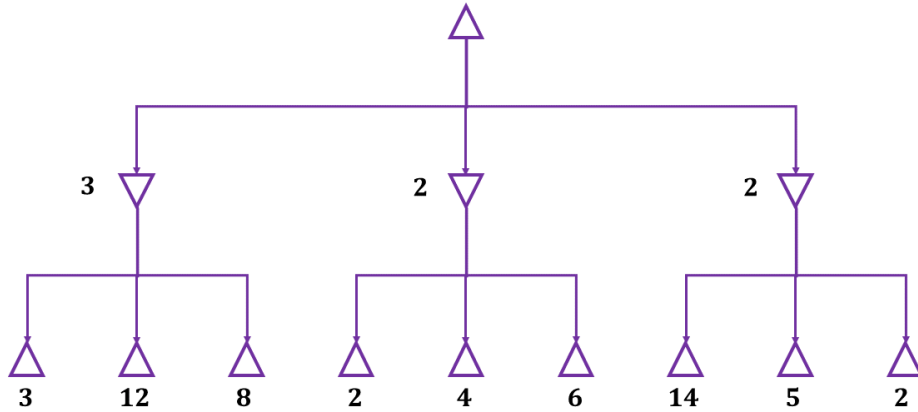
▲ المثلث الذي للأعلى يمثل اللاعب max الذي يقوم بأخذ أعلى قيمة ممكنة من الأبناء .

▼ المثلث الذي للأسفل يمثل اللاعب min الذي يقوم بأخذ أقل قيمة ممكنة من الأبناء .

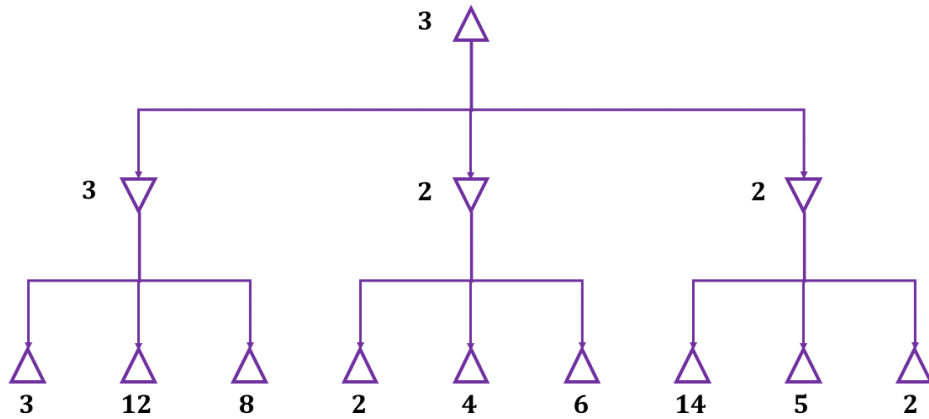
وفي الأوراق النهائية نلاحظ وجود قيم الحالات



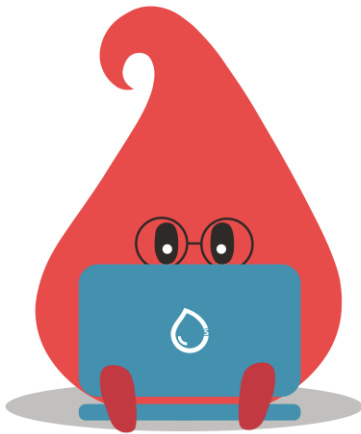
■ سنبدأ ملء المستوى الثاني من الابن الأيسر، بما أننا في حالة min سنختار الابن الأصغر وهو 3، وكذلك الابن الأوسط سيختار الابن الأصغر وهو 2، والابن الأيمن سيختار الابن الأصغر وهو 2، وهكذا تصبح القيم:



■ الآن نملأ المستوى الأول، بما أننا في حالة max نختار الابن الأكبر وهو 3:



وهكذا تكون النتيجة المتوقعة في حال لعب كلا اللاعبين بشكل أمثلي.



من سار على الدرب تعثر ونهض، خذل ووقف،

أتته لحظات يأس فحارب، ثم مشى بالعزيمة

واستعان بالله حتى وصل...

1. Complete:

الخوارزمية منتهية في حال كانت شجرة الحالات منتهية.

2. Optimal:

تعطي الحل الأمثل، لكنها تفترض أن الخصم سيلعب بشكل مثالي (في حال لم يلعب الخصم بشكل مثالي فستتمكن من الحصول على نتيجة أفضل من المتوقعة فهي ستبقى Optimal).

3. Time complexity:

تعقيدها الزمني مثل تعقيد الـ DFS وهو $O(b^m)$.

b = branching factor (عدد الحركات الممكنة)

m = depth(العمق)

4. Space complexity:

أيضاً كتعقيد الـ DFS هو $O(b * m)$.

لكن المشكلة أنه بالتأكيد من المستحيل سرد كامل فضاء البحث لمعرفة أفضل حل ممكن للعب كما نعلم، فقد يكون فضاء البحث كبير جداً

مثلاً: في الشطرنج وسطياً عدد الحركات الممكنة هو $b \approx 35$ وعدد خطوات اللعب فلنفرضه وسطياً $m \approx 100$ فتخيل حجم فضاء البحث؟! سيكون 35^{100} وفي الحقيقة إنه 10^{40} هذه النقطة تقودنا مباشرة لفكرة: يجب أن نقطع فضاء البحث + نقيم الحالات بشكل تقريبي.

للقيام بذلك نستخدم المفاهيم الثابتة التالية : cutoff test أي نوقف البحث قبل الوصول لنهاية اللعبة **لماذا؟؟؟** لأن شجرة اللعبة كبيرة جداً وليس لدينا وقت او قدرة لنحسب كل الاحتمالات لذلك نستخدم الـ depth limit لنوقف البحث عند عمق معين وبعدها نستخدم Evaluation Function لنقيم الحالة الحالية بدل ما نكمل للنهاية.



Evaluation Function or static evaluator

من الممكن في بعض الأحيان أن نتوقف عن التفرع عند عقدة ما مع أنها ليست حالة نهائية، وذلك عند الوصول إلى عمق معين مثلاً، لتجنب فرد كامل فضاء البحث والذي قد يكون كبيراً جداً، في هذه الحالة نستخدم طريقة ما (حسب اللعبة) لتقييم الرقعة الحالية تقريباً وحساب قيمة تقريبية لمدى الفائدة المتوقع أن ترجع منها، وهذا ما يقوم به تابع الـ Evaluation Function، وسيتم شرح هذه الفكرة بشكل مفصل في نهاية المحاضرة.

أولاً:

تابع الـ Evaluation Function جاء كحل لمشكلة ما هي وكيف استطاع حلها هذا ما سنشرحه الآن :

- بالعودة إلى مثال Grundy's Game لاحظ أننا قمنا بفرد جميع الاحتمالات الممكنة لدينا في اللعبة وبعدها قمنا بإعطاء قيمة للأوراق وبناء على هذه القيمة قمنا بنشر القيم من الأسفل للأعلى من أجل باقي العقد في باقي المستويات
- حسنا المشكلة أنه في بعض الألعاب مثل الشطرنج العمق غير منتهي أي عدد الحالات كبير جداً كما ذكرنا في الفقرة السابقة لذلك نحن نحتاج لحل لهذه المشكلة بحيث نستمر بفرد العقد إلى أن نصل لحد عمق معين ونقوم بإعطاء العقدة التي وصلنا لها قيمة تقديرية على الرغم أنها ليست leaf لكن نقوم بالتعامل معها على أنها كذلك.
- في مثالاً في الشطرنج في حال كان أحد اللاعبين يملك كامل القطع واللاعب الثاني يملك الملك والفيلة وبضعة بيداق فمن الواضح أن السيطرة بنسبة كبيرة للاعب الأول فمثلاً نعطيه قيمة سيطرة هي مثلاً 80%
- في لو أحضرنا لاعب محترف وقام بوضع مجموعة معايير نستطيع من خلالها تقييم الرقعة بدقة عندئذٍ لن نحتاج إلى فرد الشجرة حتى الوصول لحالة نهاية اللعبة، لأننا نملك قيمة تقريبية للرقعة الحالية
- هذه العملية من إيجاد المعايير وإعطاء قيمة لحالة ما؛ هي الوظيفة التي سنسندنها إلى الـ Evaluation function. عندئذٍ بدلاً من إكمال الرقعة حتى الحالة النهائية نعين عمقاً ما لا نريد تجاوزه وليكن max depth، وعند الوصول إلى هذا العمق نقوم بحساب الـ evaluation function للعقدة وإرجاعها وكأن هذه العقدة هي الـ leaf
- بهذه الطريقة نقل التعقيد بشكل كبير ونسرّع الخوارزمية، لكن بالمقابل لم تعد الخوارزمية مثالية not optimal
- تعتمد دقة الخوارزمية في هذه الحالة على دقة الـ Evaluation function وقيمة الـ max depth فكلما قمنا بزيادته زادت الدقة.

ملاحظة: مفهوم تابع الـ Evaluate قريب من الـ Heuristic فكلاهما يقيّم الـ State ويعتمد على الخبرة. لكن الفرق بينهما أن الـ heuristic يستخدم في إيجاد الحل فهو يتوقع الكلفة من الـ State الحالية إلى الهدف أما الـ evaluation فهو يقيّم الـ State من هو أفضل لاعب الآن وتكون القيمة لصالح max أو min.

عادةً ما نستخدم Evaluation function بحيث يرجع لنا القيم بالشكل التالي:

$f(state) \approx 0$	(تقريباً صفر) فالحالة متعادلة
$f(state) > 0$	(أكبر من الصفر) فهي جيدة بالنسبة للاعب max وسيئة للاعب min
$f(state) \gg 0$	(أكبر بكثير من الصفر) اللاعب max هو الفائز
$f(state) < 0$	جيدة بالنسبة للاعب min وسيئة بالنسبة للاعب max
$f(state) \ll 0$	اللاعب min هو الفائز

بعض الأمثلة على توابع Evaluate:

Tic-Tac-Toe

$$f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for you}]$$

ماذا نقصد بـ 3-length open؟

في $\bigcirc$ $\times$ الفوز يكون بتشكيل ثلاثة رموز إما $\times$ أو $\bigcirc$ على خط واحد:
صف - row - عمود - column - قطر - diagonal
Open يعني الخط ما فيه ولا خانة للخصم.

حسنا كيف نحسب قيمة الـ $f(n)$ هنا؟ (انتبه هذا تابع الـ evaluate)

- 1- نعدّ كم خط (صف/عمود/قطر) مفتوح إلي.
- 2- نعدّ كم خط مفتوح للخصم.
- 3- نطرحهم.

هنا بحسب منطق الدالة لدينا حالتان:

- 1 - إذا كان لدي فرص فوز مستقبلية أكثر من الخصم فالقيمة ستكون موجبة .
- 2 - إذا الخصم عنده خطوط مفتوحة أكثر فالقيمة ستكون سالبة .

لنقم الآن برسم شجرة اللعبة:

في البداية لدينا الحالتين الخاصتين:

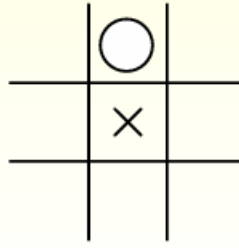
■ إذا الحالة هي فوز مؤكد لـ Max

$$f(n) = +\infty$$

■ إذا الحالة هي فوز مؤكد لـ Min

$$f(n) = -\infty$$

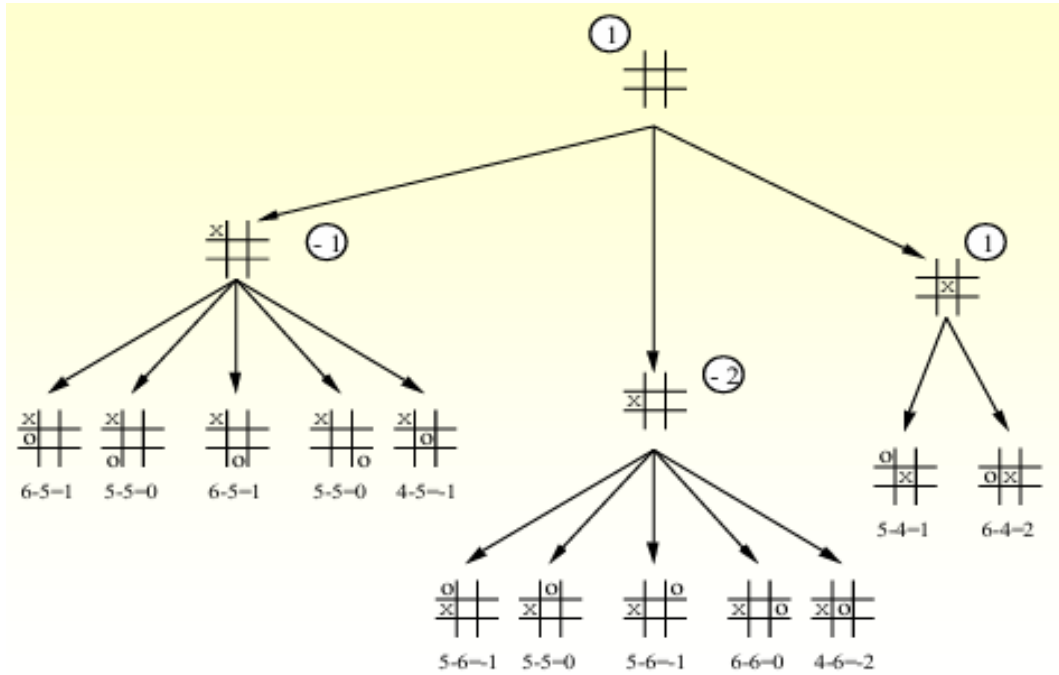
مثال قبل البدء برسم الشجرة لمعرفة كيفية حساب الاسطر والاعمدة والاقطار المتاحة لكل لاعب



- نلاحظ أن اللاعب X امامه سطرين مفتوحين وعمودين وقطرين لأنه بالوسط.
- واللعب O امامه عمودين وسطرين ولا يوجد اقطار متاحة امامه لوجود اللاعب X في المنتصف
- اذن قيمة الـ evaluate هي $4-6=2$ لهذه الحالة

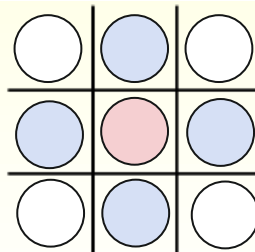
كيف علمنا أن الخط مفتوح امام اللاعب؟ يكون الخط لا يحتوي على رمز الخصم

الآن لنبدأ بشرح المخطط الشجري التالي:



نفرض أن اللاعب X هو من يبدأ باللعب :

- نبدأ من الحالة الابتدائية تكون اللوحة فارغة لا تحتوي على أي رمز والان هو دور اللاعب X للبدء باللعب
- نعلم أن عدد الأماكن المتاحة لوضع الرمز X هي 9 لكن يوجد هنا تكافؤ في الحالات حيث كل الزوايا متشابهة وكل الأطراف متشابهة والوسط حالة مستقلة كما هو موضح:



- لذلك نقوم باختصار عدد الأماكن المتاحة الى 3 بحيث X في الوسط وفي زاوية وفي طرف أي أننا لا نأخذ الأماكن المتناظرة لمنع التكرار بالشجرة.
- إذن الحالات الثلاث الأولى امام اللاعب X تكون كما هو موضح بالمخطط

الآن دور اللاعب O :

- نبدأ بالحالات من اليسار نجد أن الأماكن المتاحة أمام اللاعب O هي 8 كما هو موضح بالشكل لكن نحن (ننتبه ان لا نقوم بأخذ الحالات المتناظرة لمنع التكرار)

ملاحظة: الهدف من الشجرة توضيح كيفية حساب قيم ال evaluation وكيف تقوم ال minimax بالاختيار وليس شرح ال symmetry أي يمكن أن نجد حالات متناظرة لم يتم حذفها هذا فقط من أجل توضيح أين يمكن أن تكون قيمة التابع متساوية

الان نقوم بحساب قيمة ال evaluation لكل حالة من المستوى الأخير:

- نفرض اللاعب O هو ال min و اللاعب X هو ال max
- إذا المستوى الثاني هو دور اللاعب O الذي سيختار أسوأ قيمة بالنسبة لـ X أي اصغر قيمة من قيم الأبناء إذا سيختار 1- لعقدة اليسار وسيختار 2- لعقدة الوسط و 1 لعقدة اليمين
- وفي المستوى الأول يكون دور اللاعب X الذي سوف يقوم باختيار أسوأ قيمة لـ O أي اكبر قيمة من قيم الأبناء وهي 1

Chess

يوجد العديد من توابع ال evaluation المستخدمة ومنها:

دالة Alan Turing للشطرنج:

$$f(n) = \frac{w(n)}{b(n)}$$

w(n) : مجموع قيم قطع الأبيض

b(n) : مجموع قيم قطع الأسود

ماذا نقصد بقيم القطع ؟

Rook = 5 ♖ القلعة

Queen = 9 ♕ الملك

Pawn = 1 ♙ الجندي

Knight = 3 ♘ الحصان

Bishop = 3 ♗ الفيل

هذه القيم هي تقديرية تم وضعها من قبل خبراء لتعكس أهمية كل قطعة

■ هنا لو اللاعب الأبيض معه Queen + Rook + 4 Pawns :
 $w=9+5+4=18$

■ واللاعب الأسود معه Queen + Bishop + 2 Pawns :
 $b=9+3+2=14$

■ إذن $f=18/14 \approx 1.29$
 يعني اللاعب الأبيض متقدم.

- لو النتيجة $= > 1$ الاسود أقوى
- لو النتيجة $= < 1$ الأبيض أقوى
- لو $1 \approx$ متقاربين

لكن هذه الدالة غير كاملة لوحدها: لأن الشطرنج ليس فقط عدد قطع، في تمرکز/أمان ملك/سيطرة مربعات. لذلك نلجأ في أغلب دوال التقييم إلى مجموع مرجح (Weighted Sum Of position feature) :

$$f(n)=w_1 \cdot \text{feat}_1(n)+w_2 \cdot \text{feat}_2(n)+\dots+w_k \cdot \text{feat}_k(n)$$

أي ببساطة نحن نختار ميزات (features) تصف الوضع، مثل : عدد القطع - تمرکز القطع - سيطرة على مركز الرقعة - أمان الملك - ثم نعطي كل ميزة وزن W_i يعبر عن أهمية الميزة التي اخترناها.

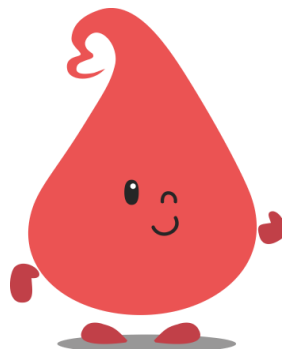
مثال:

$$f(n)=1.0(\text{material})+0.3(\text{center control})+0.5(\text{king safety})$$

وزنه أعلى $= >$ إذا material مهم جداً .
 وزنه أقل $= <$ إذا center أقل أهمية .

فائدة:

تابع الـ Deep Blue هو evaluation function يعتمد على حوالي 6000 feature يستخدم في لعبة الشطرنج ويهدف لتقليل الخطأ لأن اللعبة معقدة جداً.



ملاحظات على خوارزمية ال Minimax :

الأداء يعتمد على عاملين رئيسيين :

1) Quality of evaluation function (domain knowledge):

إذا لم نستطع الوصول لنهاية اللعبة بسبب العمق الكبير نستخدم تابع الـ Evaluation function لتقييم حالة غير نهائية إذا دالة التقييم ذكية الخوارزمية تعطي قرارات قوية وإذا الدالة ضعيفة هنا الخوارزمية يمكن أن تختار حركة قد تبدو جيدة رقمياً لكنها سيئة عملياً.

مثلاً: في لعبة O / X

إذا تابع التقييم يحسب فقط عدد الخطوط المفتوحة يمكن أن يتجاهل حالة أن يكون عند الخصم خانتين وجهاز ليكمل الثالثة.

2) Depth of search (computer power + search algorithm):

كلما زاد العمق الذي تبحث فيه تقل أخطاء تابع التقييم لأنها تقترب من نهايات حقيقية لكن العمق مكلف جداً لأن الشجرة تكبر بسرعة b^m لذلك نحتاج إلى قدرة حاسوبية وخوارزميات أسرع مثل α -beta pruning التي سنتعرف عليها بعد قليل...

Different from ordinary state space search:

1) Not to search for a solution but for one move only:

نحن نبحت عن أفضل حركة الآن فقط في خوارزميات البحث العادي نحن نبحت عن مسار للهدف.

2) No cost is associated with each arc:

في البحث العادي لدينا كلفة انتقال (مثل تكلفة طريق) وتجمعها، في الألعاب لا يوجد كلفة خطوة. الشيء المهم هو نتيجة الحالة فوز/خسارة/تعادل

Minimax Rule (Back-up values):

الهدف:

اختيار حركة واحدة لـ $\max$ تحقق :

أكبر نتيجة مضمونة لماكس حتى لو $\min$ اختار اسوء رد ممكن لـ $\max$

back up values :

قيم العقد لا تحسب مباشرة بل يتم استنتاجها من الأبناء :

عقدة $\max$: قيمة العقدة هي بقيمة أكبر قيمة من قيم الأبناء

عقدة $\min$: قيمة العقدة هي بقيمة أصغر قيمة من قيم الأبناء

Minimax procedure:

خطوات الخوارزمية:

- 1- إنشاء جذر الشجرة: الذي يمثل وضعية اللوح الحالية ومن دوره من اللاعبين.
- 2- توليد جميع الحالات الممكنة الناتجة عن حركات كل من اللاعبين.
- 3- التوقف عند توليد الحالات عند عمق محدد.
- 4- نقيم كل ورقة وصلنا لها بتابع التقييم الـ evaluation function
- 5- نرجع القيم من الأسفل للأعلى قيم العقد من قيم أبنائها Back up values
- 6- نختار الحركة التي تقود للابن الذي اعطى قيمة للجذر تعبر عن أفضل حركة للاعب الآن.

MINIMAX ALGORITHM CODE

```

1 MinMax (GamePosition game){
2   return MaxMove(game);
3}
4 MaxMove(GamePosition game ({
5   if (GameEnded (game) || DepthLimitReached()){
6     return EvalutationGameState(game, MAX)
7   } else{
8     best_move = { }
9     moves = GenerateMoves (game)
10    foreach x in moves {
11      move = MinMove(x)
12      best_move = max (best_move, move)
13    }
14    return best_move
15  }
16 }
17 MinMove(GamePosition game) {
18   if (GameEnded(game) || DepthLimitReached()){
19     return EvalutationGameState(game, Min)
20   } else {
21     best_move = { }

```

```

22 moves = GenerateMoves(game)
23 foreach x in moves {
24     move = MaxMove(x)
25     best_move = min(best_move, move)
26 }
27 return best_move
28 }
29 }

```

- نفرض أن الدور الحالي هو للاعب MAX
- نستدعي التابع MAXMOVE لاختيار أفضل حركة له.
- التابع MAXMOVE يحاول تعظيم النتيجة أي اختيار الابن الأكبر

شرط التوقف هنا:

- 1- اللعبة وصلت لعمق بحث محدد
- 2- اللعبة انتهت نستخدم تابع الـ Evaluation لتقدير الحالة يتم إرجاع قيمة الحالة الحالية

إذا اللعبة لم تنتهي اللعبة ولم نصل لعمق معين :

- 1- نولد كل الحركات الممكنة لـ max
- 2- نجرب كل حركة x :
ننتقل لحالة جديدة هي تطبيق لهذه الحركة ثم نستدعي الـ minmove لأن بعد حركة اللاعب max سيأتي دور اللاعب min نحفظ أفضل نتيجة ونعيدها أي الـ maxmove ستختار الحركة التي تعطي أكبر قيمة مضمونة لأن كل تقييم لحركة max محسوب بعد افتراض ان الـ Min سيرد بأفضل رد ضده (داخل الـ minmove)

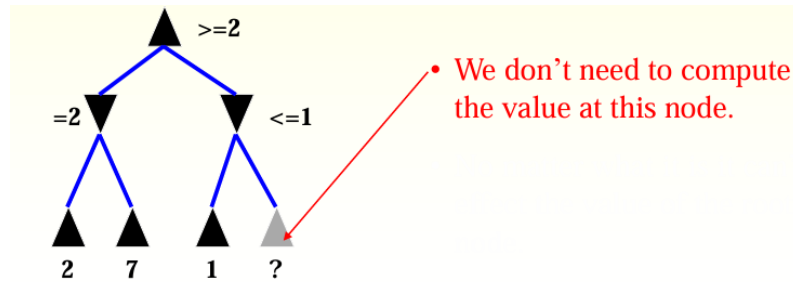
Alpha- Beta Pruning

خوارزمية alpha beta pruning كتحسين لخوارزمية الـ minimax.

ماهي المشكلة التي تقوم الخوارزمية بحلها؟

الـ minimax تقوم بحساب قيم العقد بكل الفروع حتى لو كان واضح من منتصف الطريق أن هذا الفرع لن يفيد في تغيير القرار النهائي.

ببساطة: Pat Winston "إذا عندك فكرة أكيد سيئة لا تضيع وقتك وتشوف قديش هي سيئة"



الفكرة هنا:

- إذا اكتشفنا وجود فرع يستحيل ان يكون أفضل من فرع اخر موجود نقوم بقصه prune ولا نقوم بإكمال البحث.
- هنا اللاعب بالمستوى الأول هو لاعب max أي لاعب سيقوم بأخذ أكبر قيمة من قيم الأبناء
- واللاعب في المستوى الثاني هو min أي لاعب سيقوم بأخذ اقل قيمة من قيم الأبناء اللاعب min في الفرع اليسار قيمته بقيمة الابن الأصغر له وهي 2.
- الان قيمه اللاعب ماكس ستكون 2 وباكتشاف الفرع الاخر اما ستبقى 2 او ستصبح أكبر
- نكمل اكتشاف الفرع اليمين هنا اللاعب min في الفرع لا نعرف سوى قيمة اول ابن له وهي 1 أي القيمة المبدئية لهذا اللاعب هي 1 وبما انه لاعب min فالعقدة اما تبقى 1 او ستصبح اقل اذن سواء كانت قيم ابناؤه اقل من الواحد ام أكبر فهذا لن يغير من أن قيمة اللاعب min ستكون 1 او اقل وبالتالي لن يتغير قرار اللاعب max الحتمي بأخذ قيمة اللاعب min اليسار والذي قيمته 2 والتي هي أكبر حتماً من قيمة اللاعب min اليمين.
- لذلك نتوقف عن اكتشاف أبناء اللاعب min اليمين ونقص الفرع

الفكرة العامة من المثال:

- إذا اكتشفنا أثناء البحث أن فرع مستحيل أن يعطي نتيجة أفضل من فرع آخر تم اكتشافه سابقاً فلا داعي لإكمال البحث فيه ونقوم بقصه لتوفير الوقت.
- في المستوى الأول لدينا لاعب max وهو اللاعب الذي سيختار أكبر قيمة من قيم أبنائه.
- في المستوى الثاني لدينا لاعب min وهو اللاعب الذي سيختار أصغر قيمة من قيم أبنائه.

الخطوة 1:

- نبدأ البحث باكتشاف الفرع اليسار اللاعب الموجود هنا هو min بعد اكتشاف أبنائه نجد ان أصغر قيمة لديه هي الـ 2 إذن قيمة عقدة الـ min اليسار = 2
- الآن اللاعب max يعرف ان لديه خيار يعطيه قيمة 2 وهذه القيمة أصبحت أفضل قيمة مضمونة حتى الان بالنسبة له

الخطوة 2:

- الانتقال الى الفرع الأيمن اللاعب هنا هو min أيضاً نبدأ باكتشاف اول ابن له فقط نجد ان قيمته هي 1 بما ان هذا اللاعب هو min فقيمه النهائية ستكون: 1 أو أقل من الـ 1 لان min دائماً يختار الأصغر.

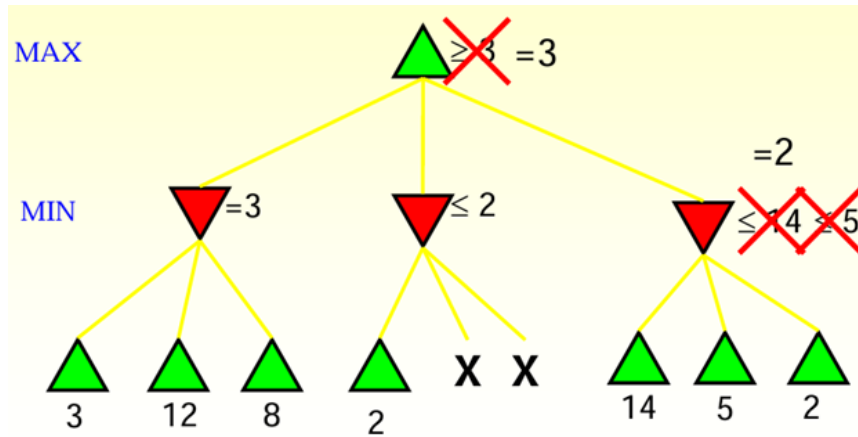
اتخاذ قرار القص pruning، اللاعب max يقارن الان:

القيمة المؤكدة من الفرع الايسر هي = 2 والقيمة المحتملة من الفرع الأيمن هي = 1 او اقل

من الواضح إذن ان 2 هي أكبر حتماً من الـ 1 او أي شيء اقل من الـ 1

إذن مهما كانت باقي أبناء الفرع الأيمن لن تتغير نتيجة اللاعب max وسيبقى اختياره هو الفرع الايسر بقيمة الـ 2

مثال 2:



بنفس أسلوب المثال السابق نحن نبحث عن أفضل قرار للاعب max مع افتراض ان اللاعب min سيلعب دائماً ضد مصلحة الـ max.

المستوى الثاني:

اللاعب min سيختار أصغر قيمة من القيم التي ستأتيه من أبنائه (عقد max)

المستوى الأول:

اللاعب max سيختار أكبر قيمة من القيم التي ستأتيه من أبنائه (عقد min)

1- الفرع الايسر:

اللاعب هنا هو min أبنائه قيمهم 8 و 12 و 3

سيقوم اللاعب باختيار أصغر قيمة وهي = 3

الان اللاعب max الجذر أصبح لديه خيار مضمون بقيمة 3

2- الفرع الأوسط:

اللاعب هنا min اول ابن له اكتشفنا قيمته هو 2 إذن قيمه هذه العقدة ستكون إما 2 او اقل

هل يوجد داعي لإكمال اكتشاف باقي الأبناء؟ لا

لماذا؟ لان اللاعب max الفرع الايسر أعطاه قيمة 3 والفرع الأوسط لن يعطيه قيمة أكبر من الـ 2 (بما انه وجد ابن بقيمة

2 فلن تكون قيمة هذا الفرع أكبر من ذلك لان اللاعب min يختار القيمة الأقل دوماً)

أي الفرع الأوسط قيمته لن تكون أفضل من قيمة الفرع الايسر التي هي 3 فلا يمكن ان يكون الخيار الأفضل لـ max

إذن نقوم بقص بقية أبناء الفرع الأوسط، ولا نكمل البحث ونعود للجذر لنأخذ الفرع الأيمن

3- الفرع الأيمن:

اللاعب min نكتشف أول ابن وهو 14 نتابع البحث ولا نتوقف لأن 14 هي أكبر من أفضل خيار مضمون للماكس إلى الآن وهو الـ 3
الابن الثاني هو 5 أيضاً نتابع البحث
الابن الثالث هو 2 نتوقف هنا لأن الـ 2 هي أصغر أبناء العقدة min والتي سيقوم اللاعب بأخذها حتماً وهي أقل من آخر خيار مضمون للماكس إلى الآن وهو الـ 3
إذن لأن قيمة اللاعب min اليمين هي 2 أيضاً فلن يقوم اللاعب ماكس باختيارها وبالتالي اللاعب max سيختار الفرع الأيسر لأن الـ 3 هي أكبر قيمة مضمونة لديه .

قاعدة خوارزمية alpha beta pruning:

1. نفرض أننا نمشي بالشجرة بشكل depth-first أي نأخذ الفرع ونمشي إلى أسفله قبل العودة واخذ فرع آخر .
2. ماذا نعني بـ ألفا α عند عقدة max :
 $\alpha(n)$ = أكبر قيمة وجدناها حتى الآن على هذا المسار لصالح MAX يعني أفضل ضمان حالياً لـ MAX
تبدأ من الـ $-\infty$ وتزيد القيمة لا تنقص
3. ماذا نعني بـ بيتا β عند العقدة min :
 $\beta(n)$ = أصغر قيمة وجدناها حتى الآن على هذا المسار لصالح min يعني أسوأ نتيجة يقدر MIN يفرضها على MAX حتى الآن.
تبدأ من الـ $+\infty$ وتنقص

نوعي القطع (Alpha cut , beta cut) :

1- Alpha cut

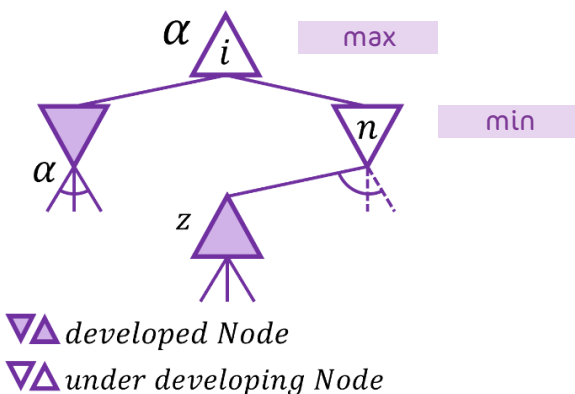
القاعدة هنا هي أن نتوقف عن البحث في فرع الـ min في حال كانت قيمة العقدة min هي أصغر أو تساوي قيمة العقدة الاب لها وهي بالتأكيد ستكون max .

$$beta(n) \leq alpha(i) \text{ أو بصيغة أخرى } z \leq \alpha$$

حيث:

n: العقدة الحالية min

i: أحد العقد الآباء ancestors من النوع max



ملاحظة:

ليس بالضرورة alpha الاب المباشر بل أي ancestor او جد او .. من النوع max

يعني إذا كنا بداخل عقدة min :

بيتا أصبحت صغيرة ولدينا قيمة العقدة ماكس الفا اكبر

== ماكس لن يختار هذا الفرع فنقوم بعمل قص من نوع alpha cutoff

2- Beta cut

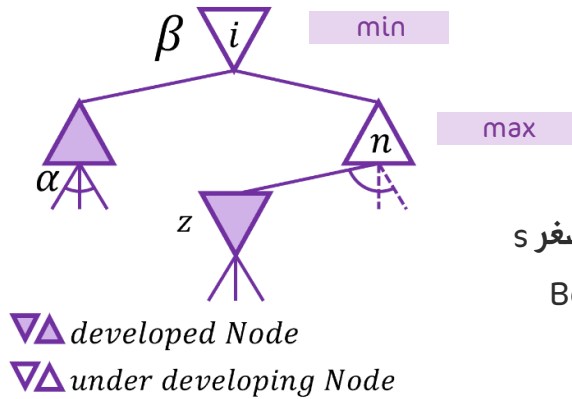
القاعدة هنا نتوقف عن البحث في فرع الـ max في حال كانت قيمة العقدة max (الفا) الحالية أكبر او تساوي قيمة بيتا لعقدة الاب الذي سيكون min

$$alpha(n) \geq beta(i) \text{ أو بصيغة أخرى } z \geq \beta$$

حيث:

n: العقدة الحالية max

i: أي عقدة الأب ancestor من النوع min



يعني إذا كنا بداخل عقدة max:

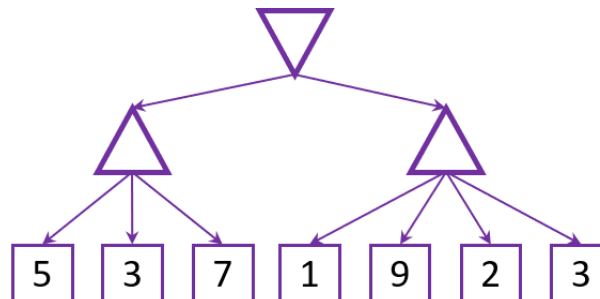
الفا أصبحت كبيرة ولدينا قيمة العقدة الاب min يلي هي بيتا اصغر s

MIN لن يسمح لـ MAX يوصل لهذه القيمة نعمل Beta cutoff

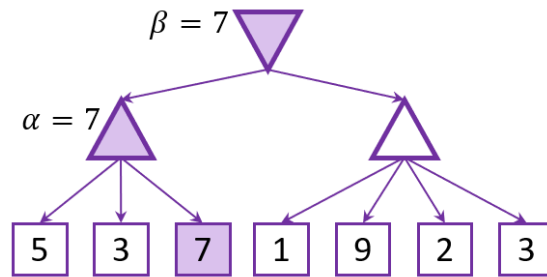
نقص تحت عقدة MAX .

α-β Pruning Example

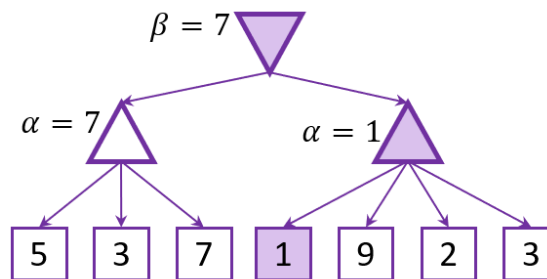
مثال 1 :



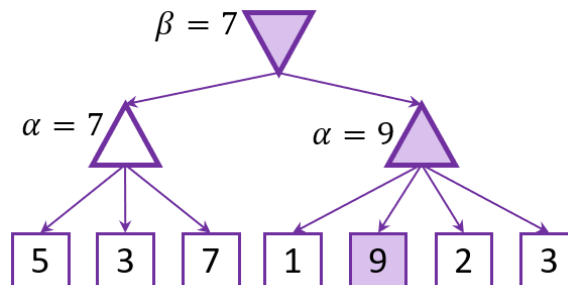
1- نبدأ بالفرع الايسر، ونجد أن قيمة الابن الأيسر هي 7، فتكون القيمة الحالية للجذر هي $\beta = 7$



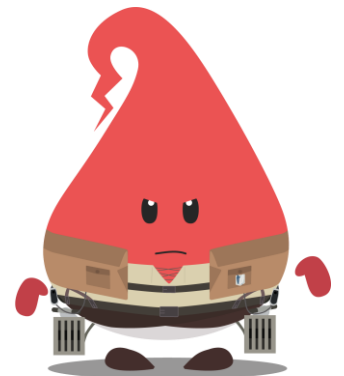
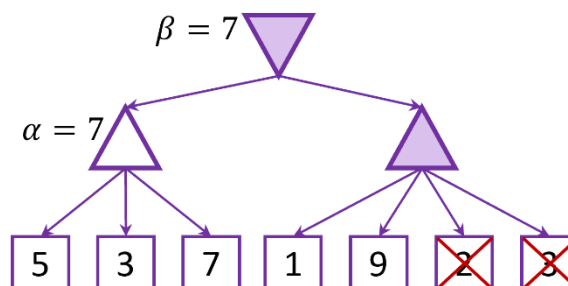
2- نذهب الى الفرع الأيمن، وتصبح قيمة الابن الايمن الحالية هي $\alpha = 1$ بما أن قيمة alpha أصغر من قيمة beta للأباء نكمل..

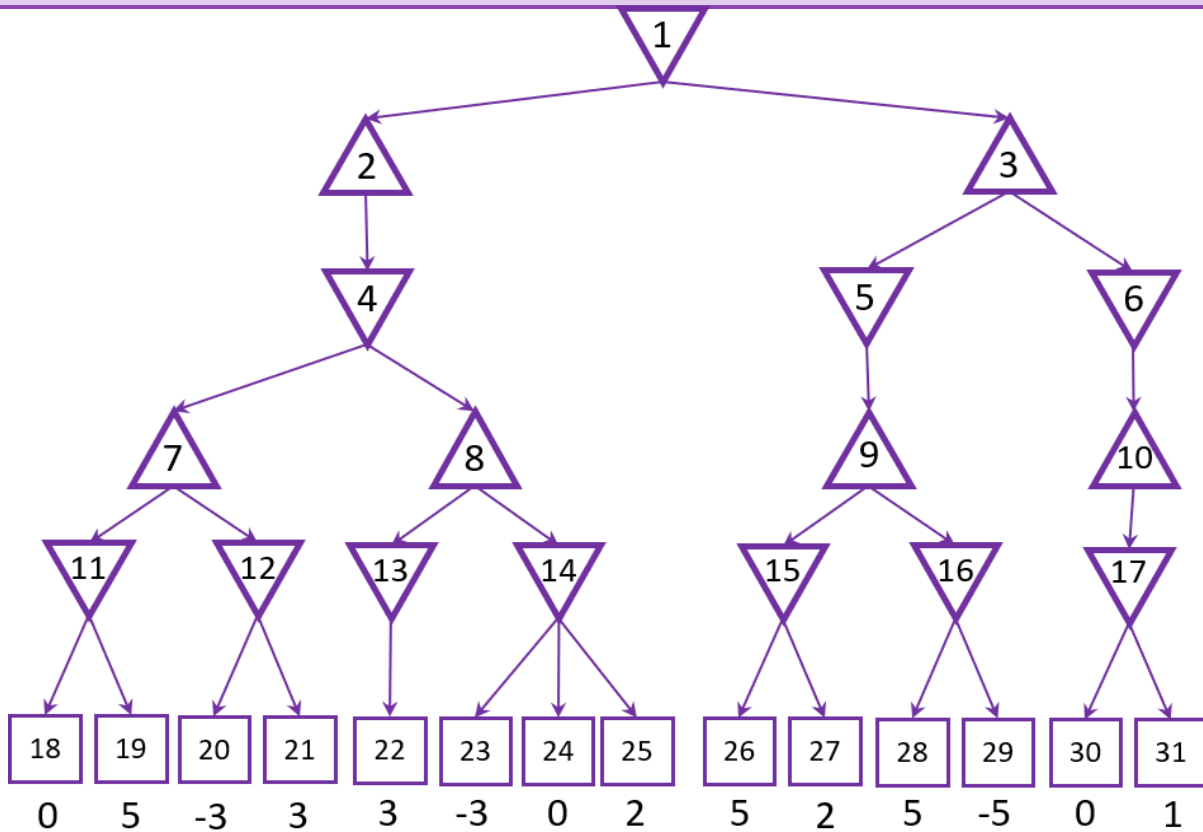


3- نجد أن العقدة $9 > 1$ فنحدّث قيمة العقدة max فتصبح: $\alpha = 9$



4- نلاحظ أن قيمة alpha أكبر من beta الأب، إذًا هذه حالة beta cut لأن: $\alpha(n) > \beta(i)$

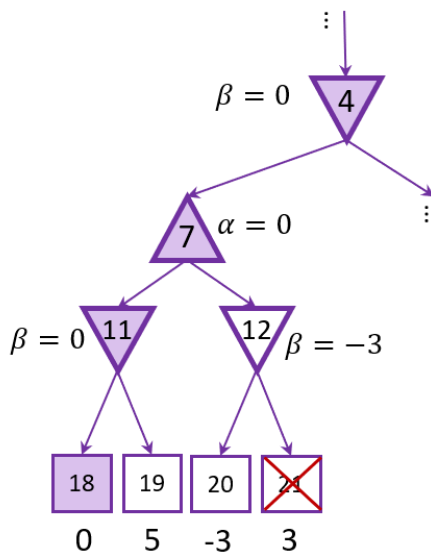




-2- في العقدة 12 نأخذ أول ابن لها 20

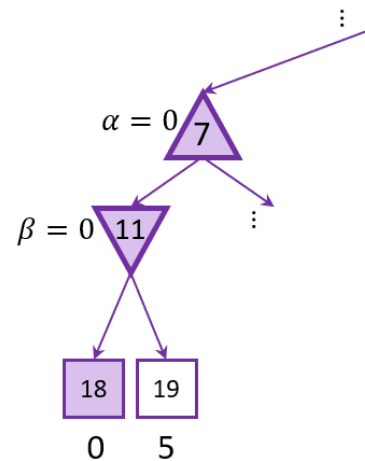
وقيمته 3- إذاً: $\beta(12) = -3$

وبما أن $\beta(12) < \alpha(7)$ هذه حالة α cut فلا نتابع معالجة أبناء العقدة 12 وتصبح قيمة العقدة 4: $\beta(4) = 0$



-1- نحسب قيمة العقدة 11 فنجد أنها 0

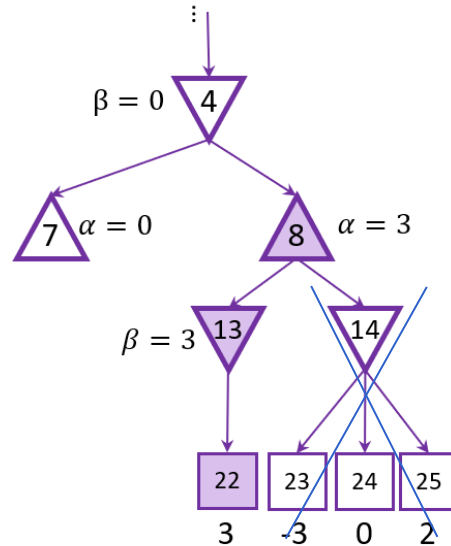
إذاً: $\beta(11) = 0 \rightarrow \alpha(7) = 0$



3- نتابع إلى العقدة 8، نحسب قيمة ابنها 13

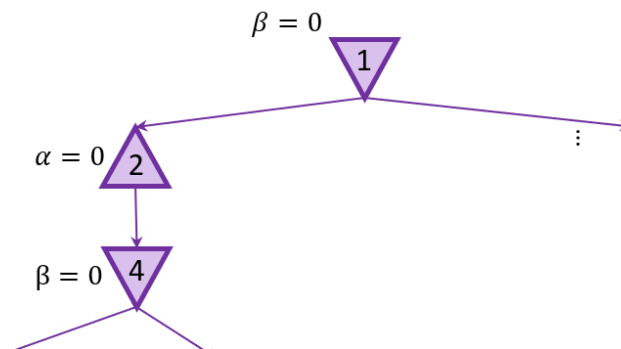
فيصبح لدينا: $\beta(13) = 3 \rightarrow \alpha(8) = 3$

نلاحظ أن $\alpha(8) > \beta(4)$ إن هذه حالة β cut فلا نتابع حساب بقية أبناء العقدة 8



إذا قيمة العقدة 4 بقيت $\beta(4) = 0$ وتصبح قيمة العقدة 2: $\alpha(2) = 0$

وأيضاً تصبح قيمة العقدة 1: $\beta(1) = 0$



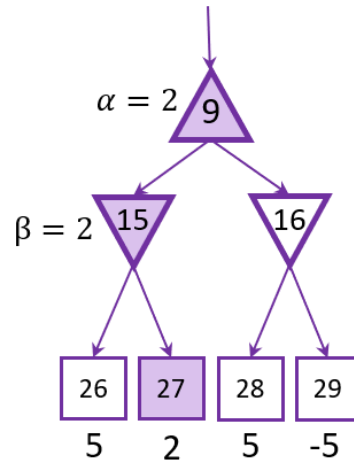
الآن ننقل إلى الفرع الثاني:

نحسب قيمة العقدة 15:

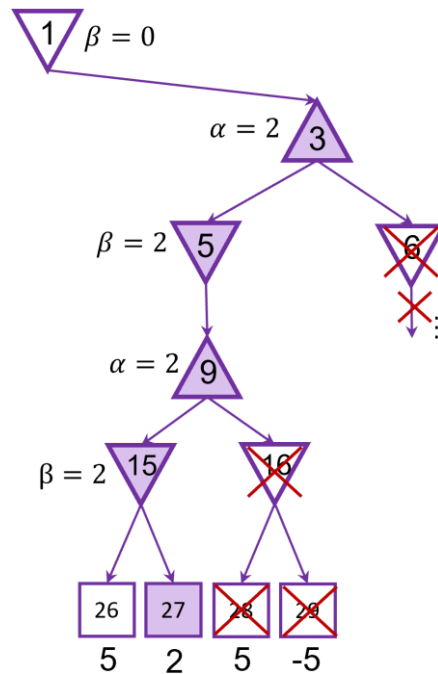
أولاً تأخذ قيمة الابن الأول العقدة 26 وقيمتها 5 أي $\beta(15) = 5$

الابن الثاني العقدة 27 قيمتها 2. أصغر من 5 فيتم تحديث القيمة $\beta(15) = 2$ ولا يوجد cut

وتصبح قيمة العقدة 9: $\alpha(9) = 2$



نلاحظ أن $\alpha(9) = 2 > \beta(1)$ إذا يوجد لدينا هنا **beta cut** فلا نعالج باقي أبناء الـ 9 إذا تصبح قيمة :
 $\alpha(3) = 2 \quad \beta(5) = 2 \quad \alpha(9) = 2$



ملاحظة هامة :

لاكتشاف وجود عملية قص لا نقوم بالمقارنة مع الاب مباشرة بالضرورة بل تتم المقارنة مع الحد الذي يفرضه أقرب خصم اعلى في الشجرة لذلك لم نقارن قيمة الفا للعقدة 9 مع قيمة بيتا لأبيها لأنه بالأساس العقدة الاب لم تحصل على قيمة لبيتا بعد وتمت المقارنة مع بيتا العقدة 1 لأنها ذات القيمة القادمة من فرع اخر تفرض المقارنة معها (القص في-Alpha Beta يعتمد على القيود المفروضة من الخصم الأعلى في الشجرة، وليس على العلاقة الأبوية المباشرة بين العقد).

نلاحظ أن:

$$\alpha(3) = 2 > \beta(1)$$

إذا أيضاً هنا **beta cut** ولا نزور باقي أبناء العقدة 3

نلاحظ أننا انتهينا بقيمة العقدة 1 هي 0

سؤال امتحاني:

في المثال السابق كم عملية CUT وكم عملية ALPHA CUT وكم عملية BETA CUT قمنا بها؟

الحل:

- 1- عدد عمليات الـ CUT الكاملة هي 4.
- 2- عدد عمليات الـ ALPHA CUT هي 1.
- 3- عدد عمليات الـ BETA CUT هي 3.

Alpha-beta pruning Code

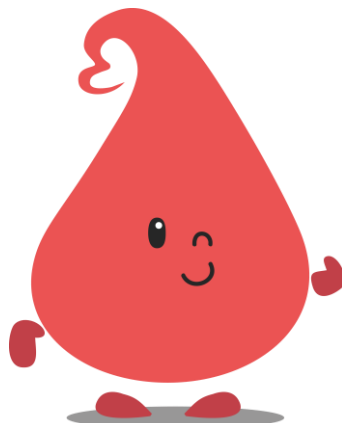
هو نفس كود خوارزمية minimax لكن مع إضافة شرط بسيط كما يلي:

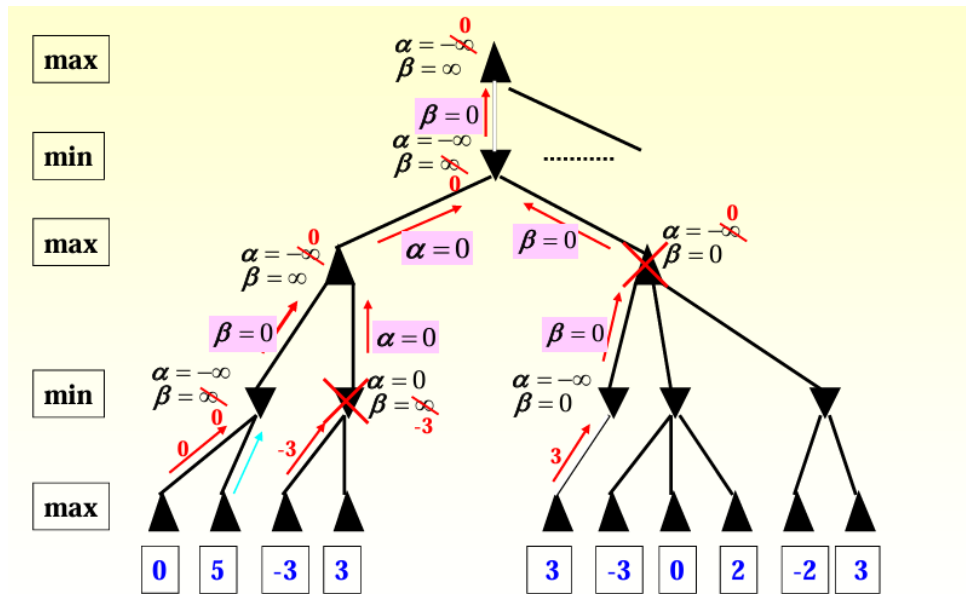
```

1 function MAX-value (n, alpha, beta)
2   if n is a leaf node then return f(n);
3   for each child m of n do
4     alpha :=max{alpha, MIN-value(m, alpha, beta)};
5     if alpha >= beta then return beta /* pruning */
6   end{do}
7 return alpha

8 function MIN-value (n, alpha, beta)
9   if n is a leaf node then return f(n);
10  for each child m of n do
11    beta :=min{beta, MAX-value(m, alpha, beta)};
12    if beta <= alpha then return alpha /* pruning */
13  end{do}
14 return beta
    
```

Initiation: $\alpha = -\infty$, $\beta = +\infty$





في هذا المثال نطبق خوارزمية Alpha-Beta Pruning على شجرة لعبة باستخدام أسلوب البحث بالعمق أولاً (DFS) الهدف هو اختيار أفضل قرار للاعب MAX مع افتراض أن اللاعب min سيلعب دائماً بأفضل طريقة ضده. الفكرة الأساسية للقص (Pruning) هي التوقف عن استكشاف الفروع التي ثبت أنها لن تغير القرار النهائي.

1. البدء من الجذر (MAX)

الجذر يمثل اللاعب MAX وهدف MAX هو اختيار أكبر قيمة من قيم أبنائه. نبدأ الاستكشاف من الفرع الأيسر لأننا نتبع DFS

2. دخول الفرع الأيسر (MIN)

هذه العقدة تمثل اللاعب MIN سيختار في النهاية أصغر قيمة من قيم أبنائه. في البداية تكون الحدود:

$$\alpha = -\infty, \beta = +\infty$$

نتابع النزول إلى الابن الأيسر (MAX) ثم إلى العمق.

3. تقييم الأوراق الأولى

أول ورقة لها قيمة 0 والورقة الثانية لها قيمة 5 بما أن العقدة المعنية هنا ستقيم من منظور MIN في هذا المسار، فالقيمة المختارة تكون 0 الأصغر

4. الرجوع إلى عقدة MAX أعلى

نعود إلى عقدة MAX الأب ومعنا القيمة 0 يتم تحديث قيمة ألفا كانت $\alpha = -\infty$ أصبحت $\alpha = \max(-\infty, 0) = 0$ الآن MAX لديه خيار مضمون بقيمة 0

5. متابعة الفرع الآخر داخل نفس المستوى

ننتقل إلى الفرع اليميني تحت عقدة MAX نصل إلى ورقة بقيمة -3
هذا يعني أن عقدة MIN في هذا الفرع قادرة على فرض قيمة ≤ -3
وبما أن MAX في الأعلى لديه بالفعل خيار بقيمة 0، فإن هذا الفرع لن يتفوق على الخيار الحالي.
النتيجة: قصّ (Prune) بقية الأبناء في هذا الفرع وعدم إكمال الاستكشاف.

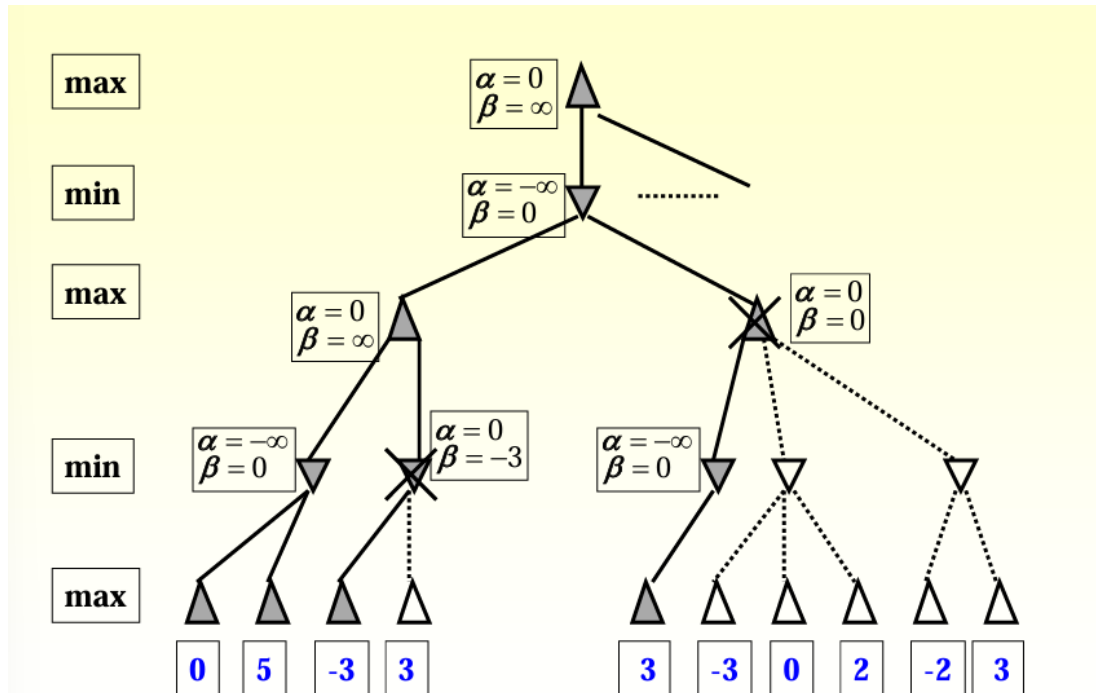
6. الرجوع إلى عقدة MIN الأعلى وتحديث بيتا

نعود إلى عقدة MIN الأعلى ومعنا القيمة 0 يتم تحديث قيمة بيتا كانت $\beta = +\infty$ صارت $\beta = \min(+\infty, 0) = 0$
هذا يعني أن MIN أصبح لديه حدّ أفضل ضد MAX بقيمة 0

7. الانتقال إلى الفرع اليميني من عقدة MIN

ننتقل إلى الابن اليميني (MAX) ننزل إلى أول ورقة في هذا الفرع ونجد قيمتها 3 يتم تحديث ألفا في عقدة MAX لتصبح $(\alpha = 3)$
الآن لدينا $(\alpha = 3)$ $(\beta = 0)$ حدّ مفروض من عقدة min أعلى
بما أن هذا الحد يبيّن أن متابعة بقية الأبناء لن تغيّر القرار النهائي، يتم قصّ باقي الفروع.

النتيجة النهائية تم استكشاف فقط الفروع الضرورية لاتخاذ القرار. تم قصّ الفروع التي ثبت مبكراً أنها لن تؤثر على اختيار MAX، القيمة النهائية التي يعتمد عليها الجذر هي 0
هذا هو شكل الشجرة بعد عمليات القص :



Properties of α - β

خصائص خوارزمية الـ Alpha Beta Pruning :

1- عملية القص لا تؤثر على النتيجة النهائية:

أي الـ alpha beta pruning لا تغير قرار الـ minimax
ستبقى الحركة نفسها ونفس القيمة الفرق الوحيد أننا نحسب قيم لعقد أقل أي فائدة هذه الخوارزمية هي اختصار الوقت

2- ترتيب الحركات هنا مهم جداً:

إذا اكتشفنا الحركات الجيدة أولاً ثم السيئة فإن عملية القص تحدث بشكل أبكر وبالتالي يقل عدد العقد المحسوبة بينما إذا بدأنا بالحركات السيئة فالقص سيكون متأخر وبالتالي يزيد عدد العقد المحسوبة.
إذاً ترتيب توليد الحركات يصنع الفرق في كفاءة الخوارزمية .

3- لو افترضنا ان الترتيب للحركات هو الترتيب المثالي عندها التعقيد الزمني سيتحسن من

$$O(b^{m/2}) \leq O(b^m)$$

كأننا ضاعفنا عمق البحث بنفس الوقت والقدرة

4- Meta reasoning :

في هذه الخوارزمية نحن نفكر بـ ما الذي يستحق ان نحسبه وما الذي لا يستحق بدل ان نحسب كل شيء
نقول هذا الحساب لن يغير القرار إذا لا داعي له وهذا ما يسمى بالـ Metareasoning أي التفكير حول التفكير نفسه.

Deterministic games in practice (الألعاب الحتمية في الواقع):

الان السؤال الحقيقي هو هل هذه الخوارزميات مجرد نظريات ام انها نجحت بالفعل؟

Checkers

في لعبة الداما (Checkers) ، سنة 1994، برنامج اسمه Chinook استطاع أن يهزم بطل العالم البشري Marion Tinsley بعد سيطرة بشرية دامت 40 سنة.
لأن الداما لعبة حتمية ومعلوماتها كاملة وعدد الحالات الممكنة فيها أصغر نسبياً من الشطرنج فاستطاعوا يبنوا قاعدة بيانات نهايات اللعبة أي كل وضعية فيها 8 قطع أو أقل، البرنامج يعرف اللعب المثالي فيها 100%
عدد هذه الحالات كان ضخم (مئات المليارات)، لكنه محسوب مسبقاً.

■ Chess

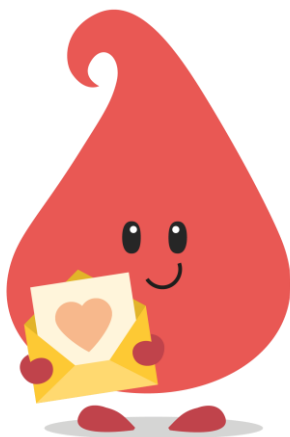
في الشطرنج، سنة 1997، برنامج Deep Blue هزم بطل العالم غاري كاسباروف.
كيف؟

كان Deep Blue:

- يبحث حوالي 200 مليون وضعية في الثانية
 - يستخدم Evaluation Function معقدة جداً
 - ويستعمل Alpha-Beta pruning لتقليل عدد الفروع
- أحياناً يمدّ البحث حتى 40 حركة للأمام (ply) في بعض الخطوط المهمة

■ Othello

في لعبة Othello الحواسيب أصبحت أفضل بكثير من البشر لدرجة أن أبطال البشر يرفضون المنافسة



Finally, the end